SECUREQUIZ

Project Roadmap & Completion Handbook

A clear, test-gated plan from first requirements to a deployed MVP

DOCUMENT PURPOSE	A detailed implementation and verification record for the browser-first SecureQuiz examination platform.
COMPANION TRACKER	SecureQuiz_Living_Project_Tracker_v2.xlsx — use it to mark progress, record test evidence, and track decisions.
SCOPE	12 phases, 110 implementation tasks, 24 baseline test scenarios, phase exit gates, and a post-MVP backlog.
INITIAL STATUS	All implementation tasks are unchecked. Nothing is marked complete on your behalf.
VERSION DATE	17 October 2026

The governing rule

A task is complete only when the work is implemented, its test passes, and its acceptance criterion is met. A page that looks finished is not enough if the backend rules or failure cases are still untested.

Security boundary

SecureQuiz can collect browser-observable events and combine presence signals, but a normal browser cannot guarantee prevention of cheating or prove physical location. The platform must show evidence and uncertainty honestly, and let teachers review incidents.

How to use this handbook

Use the PDF as the readable project reference and the spreadsheet as the day-to-day progress record. They are designed to work together.

1. Work in sequence

Complete the current phase before moving to the next, unless a documented dependency requires a small parallel task.

2. Keep the task IDs

IDs such as P05-07 identify the same task in the PDF and the Roadmap sheet. Use IDs in notes, commits, and questions.

3. Test before ticking

Run the test written for the task, inspect the actual result, and verify the acceptance gate. If any part fails, use In progress.

4. Record evidence

Write a test name/result, commit hash, pull request, screenshot reference, or deployment check. Never put passwords or private tokens in notes.

5. Keep scope under control

Add new ideas to Feature Backlog. Do not let optional AI, camera, BLE, or native-app work block the MVP.

6. Update the docs

When architecture, database fields, security rules, or product scope changes, update the corresponding docs and Decision Log.

Status meanings

Status	Meaning
☐ Not started	No verified implementation work yet.
◐ In progress	Work has started, but the test or acceptance gate is not yet satisfied.
☑ Done	Implementation, test, and acceptance gate all pass; evidence is recorded.
▯ Deferred	Optional feature deliberately postponed; it does not block the current release.

Recommended work loop

Read task → implement the named scope → test the behaviour → run relevant regression tests → compare actual result with acceptance gate → record evidence → update status → continue.

1. Product definition and scope

Product: SecureQuiz is a responsive, browser-first examination platform. Teachers create and configure quizzes, invite eligible students, monitor the live session, review supported security signals, and access results. Students join through an invitation, complete entry checks, answer questions, and submit. The backend owns authorization, deadlines, answer validation, grading, risk scoring, and audit records.

Primary user roles

Role	Responsibilities	Critical restrictions
Teacher	Create/manage own quizzes; schedule/start/end exams; monitor participants; review incidents; grade/publish results.	Cannot access another teacher's private exam unless explicitly authorized.
Student	Join eligible exam; view allowed questions; save own answers; submit own attempt; see permitted results.	Cannot access other attempts, answer keys, or teacher-only monitoring data.
Administrator	Manage institution-level accounts/policies and operational records where required.	Admin powers must be explicit, limited, and audited.

MVP scope

☐ Teacher authentication and server-side role/ownership checks.

☐ Classrooms and student eligibility/roster handling.

☐ Quiz authoring with single-answer MCQs, marks, options, and private answer keys.

☐ Random, expiring, revocable invitation links and a waiting room.

☐ Exam attempt lifecycle, server-authoritative deadline, answer autosave, and safe submission.

☐ Browser-observable event logging for supported visibility, focus, fullscreen, and clipboard interactions.

☐ Backend-validated/deduplicated events, explainable rule-based risk assessments, and a live teacher dashboard.

☐ Short-lived classroom QR challenge with teacher confirmation/exception fallback.

☐ Automatic MCQ grading, results, CSV export, incident reports, and audit records.

☐ Testing, supported-browser documentation, deployment, and recovery procedures.

Explicit non-goals for MVP

☐ Guaranteed cheating prevention or automatic determination that a student cheated.

☐ Reliable detection of every new tab, other application, screenshot, second device, or external assistance.

☐ Browser-only Wi-Fi SSID discovery or exact physical-location proof.

☐ Universal Bluetooth scanning from ordinary Android browsers.

☐ Continuous webcam/microphone monitoring or AI-based cheating classification.

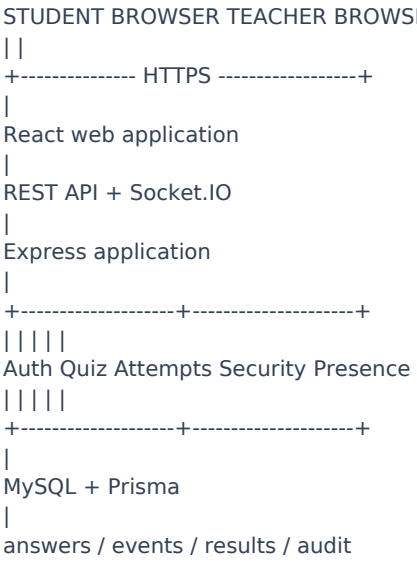
☐ A separate native Android app before the browser workflow is reliable.

2. Architecture and technology decisions

Recommended stack

Layer	Choice	Reason / notes
Frontend	React + Vite; Tailwind CSS; React Router; Axios	Responsive web client; teacher and student views share core components.
Backend	Node.js + Express, modular monolith	One deployable backend with domain modules; avoid premature microservices.
Database	MySQL + Prisma	Relational ownership and exam relationships, foreign keys, migrations, transactions.
Realtime	Socket.IO	Live monitoring and reconnection; every room subscription must be authorized.
Validation	Zod or Joi	Validate request bodies, query parameters, and settings at the API boundary.
Authentication	Argon2id password hashing; short-lived access and secure session/refresh design	For cookie sessions, use HttpOnly, Secure in production, suitable SameSite, and CSRF protection.
Testing	Vitest, Supertest, Playwright; real Android browser checks	Unit, API integration, end-to-end, and device compatibility testing.
Deployment	Render plus a managed MySQL provider as an initial option	Verify current pricing, connection limits, backups, and operational requirements before selecting providers.

Logical architecture



Backend module boundaries

Module	Owns
Auth/users	Identity, sessions, account status, role checks.
Classrooms	Class groups, roster, memberships, eligibility.
Quizzes/questions	Quiz configuration, question versions, options, server-only answer keys.
Invitations/exam sessions	Join token lifecycle and session state transitions.
Attempts/answers/submissions	Attempt state, server deadline, answer validation, finalization.
Security/risk assessment	Event validation, deduplication, policy version, score and explanations.
Presence	Short-lived challenges, verification outcomes, manual fallback.
Monitoring/teacher actions	Live participant state, incident timeline, warnings/termination.
Results/audit/reports	Grading, result publication, exports, action history.

Architecture rule

Keep business rules in backend services, authorization on every protected API and socket room, and database constraints for important invariants. The frontend presents state; it does not own authoritative state.

3. Examination and attempt state machines

The server must enforce valid state transitions. Frontend buttons are not security controls; every requested transition must be validated in the backend and recorded where important.

State	Meaning	Typical allowed next state(s)
DRAFT	Teacher is still editing.	SCHEDULED / LOBBY_OPEN, depending on design.
SCHEDULED	Published and awaiting the configured start window.	LOBBY_OPEN / CANCELLED.
LOBBY_OPEN	Eligible students may complete entry checks.	IN_PROGRESS / CANCELLED.
IN_PROGRESS	Students may answer within server policy.	PAUSED / ENDED.
PAUSED	Teacher has temporarily paused the session.	IN_PROGRESS / ENDED.
ENDED	No new answers unless an explicit exception policy applies.	RESULTS_PUBLISHED.
RESULTS_PUBLISHED	Results are available to permitted users.	Usually terminal; corrections are separately audited.

Attempt states

State	Rule
CREATED	Attempt exists but has not started answering.
READY	Required entry checks passed and attempt is ready.
IN_PROGRESS	Answers can be changed according to policy until the server deadline.
SUBMITTED	Student's final submission accepted.
AUTO_SUBMITTED	Server finalized the attempt at deadline according to policy.
ABANDONED	Attempt ended without a valid submission under documented policy.
INVALIDATED	Authorized action invalidated attempt; actor, time, and reason are recorded.

Rules to decide and document

☐ Whether the timer begins at the teacher's start or when each student begins.

☐ Whether pausing freezes every student's timer or only prevents new answers; make this explicit.

☐ Whether reconnecting students receive a grace period and how late queued answers are handled.

☐ Whether a teacher can extend a deadline, and how the change is recorded.

☐ How concurrent start/submit requests are handled atomically.

☐ Whether submitted answers can ever be changed; if so, use a controlled, audited correction process.

4. Database design and integrity rules

Create the schema incrementally as phases introduce the relevant features. Use migrations, foreign keys, uniqueness constraints, and transactions where a set of related changes must succeed together.

Entity	Purpose / important fields
institutions	Optional tenant boundary; institution name, status, timestamps.
users	Identity, email, password hash, role, status, institution reference.
classrooms	Institution, teacher/owner, name, status.
classroom_memberships	Classroom/student relation; prevent duplicate membership.
quizzes	Owner, title, duration, availability, status, randomization, policy reference.
questions / question_options	Question type, content, marks, order, options; answer key must not be exposed to students.
quiz_invitations	Hash of random token, expiry, revocation, permitted use and associated quiz/session.
exam_sessions	One scheduled/running instance of a quiz; state, start/end and policy snapshot.
exam_participants	Eligibility, participant status, session link.
attempts	Attempt number, status, start time, server deadline, submission and activity timestamps.
answers	Attempt/question, answer payload, version, save time; use versioning for stale writes.
presence_challenges / verifications	Challenge expiry/replay policy and method-specific outcome.
security_events	Attempt, client event UUID, event type, limited metadata, server receipt time, policy version.
risk_assessments	Calculated risk level/score and reasons derived from accepted events.
teacher_actions	Actor, target attempt, action, timestamp, reason, related incident.
submissions / results	Finalized answers, score, grading state, result publication state.
audit_logs	High-value actions and state changes; access restricted and append-only by application policy.

Database invariants to test

☐ A student's answer must reference a question belonging to the active attempt's exam version.

☐ A student cannot exceed the permitted attempt count.

☐ An attempt cannot be finalized twice into two different outcomes.

☐ A revoked/expired invitation or presence challenge cannot be accepted.

☐ A repeated security-event UUID cannot increase the score again.

☐ A teacher cannot modify another teacher's resources without explicit authorization.

☐ A submitted attempt cannot accept ordinary answer updates.

☐ Server deadlines and grading results cannot be overridden by client-supplied values.

☐ If a quiz can be edited after publication, preserve the question/policy version used by each attempt.

5. API planning reference

The routes below are illustrative API groups, not code to copy blindly. Keep a consistent version prefix, validate every request, enforce ownership, and document response/error formats in the API specification.

API group	Responsibilities / example operations
Auth	Register/login/refresh/logout/current user; session revocation.
Classrooms	Create/list/view/update/archive; roster import; membership add/remove.
Quizzes/questions	Create/update/preview/publish; question and option management.
Invitations/sessions	Create/revoke invitation; create/start/pause/end exam session.
Join/participants	Validate invitation; verify eligibility; enter waiting room; participant status.
Attempts/answers	Get current attempt; deliver allowed questions; save answer; submit; recover state.
Security	Ingest accepted event; fetch authorized incident timeline; fetch session summary.
Presence	Create challenge; verify challenge; read verification summary; record teacher confirmation.
Monitoring	Read authorized participant snapshot; receive live events; perform audited teacher action.
Results/reports	Read authorized results; publish; export CSV; generate incident report.

API acceptance checklist

☐ Every protected route has authentication and authorization tests.

☐ Every request body, query, and route parameter is validated.

☐ Errors have consistent status codes and do not leak stack traces or secrets.

☐ The backend derives user identity from the authenticated session, not a client-supplied user ID.

☐ Rate limits protect sensitive operations without preventing legitimate answer submission.

☐ Request IDs or idempotency/version rules protect retryable operations.

☐ Student-facing DTOs omit answer keys, internal risk rules where inappropriate, and other students' data.

6. Presence verification and browser monitoring

Presence: start with a short-lived QR challenge and teacher fallback. The browser generally cannot reliably reveal the Wi-Fi SSID or universally scan BLE beacons. Network-side integration requires institution infrastructure and permission; BLE may require a supported native app later.

Method	Useful as	Limitations / handling
Roster/enrollment	Eligibility check	Typed name/roll number alone is not identity proof.
Short-lived QR	Current challenge response tied to exam session	Can be photographed/shared; expire and rotate; do not claim exact location proof.
Teacher confirmation	Human fallback/exception review	Record who confirmed, when, and why if needed.
Managed network integration	Network-path context when institution cooperates	IP/network path alone does not prove classroom presence.
BLE via native app later	Additional proximity signal	Browser support is limited; spoofing/relay and distance uncertainty remain.

Browser events and risk scoring

Event/signal	Suggested treatment	Never assume
Page becomes hidden/visible	Record supported event with server receipt time; apply conservative policy.	That a new tab or cheating was definitively observed.
Window focus changes	Use as a weaker contextual signal.	That focus loss proves another app was used for cheating.
Fullscreen exit	Record if fullscreen was required; show clear warning/fallback.	That fullscreen locks the operating system.
Copy/cut/paste interaction	Block supported events in the exam UI if policy requires; consider accessibility.	That all copying or external help is prevented.
Connection loss/reconnect	Track connection health and save/recovery state separately.	That network problems should increase cheating risk.
Presence verification failure	Show separate presence state and exception workflow.	That failure itself proves misconduct.

Risk engine rules

☐ The browser submits event type and limited metadata; the backend chooses accepted severity/points.

☐ Never accept client-supplied risk score or severity as authoritative.

☐ Deduplicate by event UUID and enforce event-frequency/payload limits.

☐ Keep the accepted event history and policy version so a score can be explained later.

☐ Keep risk, connection health, and presence status as separate dimensions.

☐ Use teacher review before serious consequences; a score is not proof of cheating.

7. Real-time monitoring and answer reliability

Real-time event flow

☐ Student browser emits a supported event or answer update.

☐ Backend authenticates and validates the request.

☐ Backend checks attempt/session state and deduplicates retryable events.

☐ Backend persists accepted state before broadcasting important updates.

☐ Socket.IO publishes only to authorized rooms.

☐ Teacher dashboard updates the row and allows opening the incident timeline.

☐ On refresh/reconnect, the dashboard rebuilds from an authorized server snapshot.

Socket.IO authorization checklist

☐ Authenticate the socket connection.

☐ Verify teacher ownership or student participation before joining a room.

☐ Never treat a room name/session ID as authorization.

☐ Send students only their own permitted information and shared exam instructions.

☐ Handle reconnect and stale sockets; do not rely on socket history as the source of truth.

☐ If scaling to multiple backend instances later, plan a shared adapter and cross-instance state carefully.

Autosave and recovery checklist

☐ The frontend shows Saved only after backend acknowledgement.

☐ Use request IDs or per-question versions to avoid stale requests overwriting newer answers.

☐ IndexedDB can help recover pending drafts but is not authoritative evidence.

☐ On reconnect, fetch server state and reconcile pending local changes with explicit conflict rules.

☐ The server applies the deadline/late-answer policy to queued changes.

☐ Test database failure, backend restart, browser refresh, offline/online transitions, and duplicate submission.

8. Teacher dashboard and incident workflow

Column	What it communicates
Student / roll	Identity in the class roster; only visible to authorized teacher.
Attempt status	Waiting, ready, active, submitted, or other documented state.
Connection	Connected, reconnecting, disconnected, plus last-seen where useful.
Presence	Unverified, QR verified, teacher confirmed, or exception review.
Progress	Answered/total; avoid exposing answer content to other students.
Risk level	Policy-derived indicator plus count/type of accepted events.
Last activity	Server-received event or saved-answer time.
Action	Open timeline, send warning, or end attempt if policy/authorization allows.

Incident timeline should show

☐ Server-received timestamps and event types.

☐ Which security-policy version was used.

☐ The risk score before/after or a clear list of contributing events.

☐ Connection interruptions as operational events, not cheating events by default.

☐ Presence verification method/outcome, with limitations.

☐ Warnings, manual review decisions, attempt termination, and stated reason.

☐ A clear distinction between observed fact and interpretation.

Privacy and retention

☐ Collect only identity, answer, event, and operational data needed for stated purposes.

☐ Do not collect continuous location, microphone, webcam, or extra device identifiers by default.

☐ Document who can read/export answers, events, reports, and audit records.

☐ Set retention/deletion rules with institutional requirements and applicable law in mind.

☐ Treat any future camera/microphone feature as a separate privacy/security review, not a casual extension.

☐ Keep secrets and unnecessary personal data out of application logs and progress notes.

P00 | Project control, scope and definition

Goal: Lock the first release scope, define what SecureQuiz can and cannot claim, and establish a repeatable build-test-accept cycle.

Primary file/folder touchpoints:
docs/requirements.md
docs/scope-and-non-goals.md
docs/architecture.md
docs/threat-model.md
docs/decision-log.md
docs/test-strategy.md
README.md

Checklist: 10 tasks

☐ P00-01 | Create the project repository and working branches

What to do: Create the Git repository, main branch, feature-branch convention, and a clear commit-message convention.

Folders / files: Repository root; README.md; .gitignore

How to test: Clone the repository into a clean directory and confirm the working tree is clean.

Acceptance gate: Repository exists, has a README and .gitignore, and can be cloned.

Evidence / notes: _____

☐ P00-02 | Write the project problem statement

What to do: Describe the users, classroom examination problem, intended value, and what the product will deliver.

Folders / files: docs/requirements.md

How to test: Ask someone unfamiliar with the project to explain the product after reading the statement.

Acceptance gate: Problem, users, and intended outcome are clear in one page.

Evidence / notes: _____

☐ P00-03 | Define MVP and non-goals

What to do: MVP: teacher accounts, quizzes, MCQs, invitation links, student attempts, autosave, timer, event logging, risk assessment, live dashboard, grading, QR presence challenge. Non-goals: guaranteed cheating prevention, AI detection, BLE scanning in ordinary browsers, webcam recording, native app.

Folders / files: docs/scope-and-non-goals.md

How to test: Review every planned feature and mark it MVP, later, or explicitly out of scope.

Acceptance gate: No ambiguous feature is required to finish the first release.

Evidence / notes: _____

☐ P00-04 | Define browser-security claims accurately

What to do: Document that browser telemetry is incomplete and fullscreen/clipboard restrictions are deterrents, not a secure device lockdown.

Folders / files: docs/security-model.md

How to test: Review every UI label and product claim for overpromising.

Acceptance gate: No feature claims to prove cheating or physical presence from a weak signal.

Evidence / notes: _____

☐ P00-05 | Define roles and permissions

What to do: Define TEACHER, STUDENT, ADMIN permissions, ownership rules, and institution/classroom boundaries.

Folders / files: docs/requirements.md; docs/security-model.md

How to test: Review a role-permission matrix with examples of allowed and forbidden actions.

Acceptance gate: Every protected resource has an owner/authorization rule.

Evidence / notes: _____

☐ P00-06 | Define examination state machine

What to do: Define quiz and exam-session states, allowed transitions, who can perform them, and behaviour on pause/end/cancel.

Folders / files: docs/exam-state-machine.md

How to test: Walk through valid and invalid transitions on paper.

Acceptance gate: Every transition has a rule, actor, and expected failure behaviour.

Evidence / notes: _____

☐ P00-07 | Define attempt state machine

What to do: Define CREATED, READY, IN_PROGRESS, SUBMITTED, AUTO_SUBMITTED, ABANDONED, INVALIDATED as needed; define reconnection and duplicate submission behaviour.

Folders / files: docs/attempt-state-machine.md

How to test: Simulate refresh, reconnect, late submit, and duplicate submit scenarios.

Acceptance gate: No ambiguous attempt transition remains.

Evidence / notes: _____

☐ P00-08 | Define success criteria and release gates

What to do: Create measurable acceptance criteria for core workflows, security, reliability, browser support, and deployment.

Folders / files: docs/release-checklist.md

How to test: A second person can decide objectively whether each criterion passes.

Acceptance gate: Every phase has a pass/fail gate before the next phase starts.

Evidence / notes: _____

☐ P00-09 | Create decision log

What to do: Record stack choices, why they were chosen, alternatives rejected, and dates of changes.

Folders / files: docs/decision-log.md

How to test: Add and review the initial architecture decisions.

Acceptance gate: Major decisions are recorded rather than buried in chat history.

Evidence / notes: _____

☐ P00-10 | Set up project documentation index

What to do: Create a single index linking requirements, architecture, database, APIs, threat model, tests, and deployment docs.

Folders / files: docs/README.md; README.md

How to test: Open every link and verify it resolves.

Acceptance gate: Documentation can be navigated from the repository root.

Evidence / notes: _____

PHASE EXIT GATE

☐ **Scope, non-goals, roles, state machines, and release criteria are documented and understood.**

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P01 | Architecture and repository foundation

Goal: Create the frontend, backend, database setup, environment conventions, and initial automated checks.

Primary file/folder touchpoints:
frontend/src/{components,pages,layouts,hooks,services,features,utils}
backend/src/{config,modules,middleware,sockets,jobs,utils}
backend/prisma/schema.prisma
docs/architecture.md
tests/{integration,e2e,security}
.env.example
docker-compose.yml (optional local setup)

Checklist: 9 tasks

☐ P01-01 | Choose and record the initial stack

What to do: Use React + Vite, Tailwind CSS, React Router, Axios, Node.js + Express, MySQL + Prisma, Socket.IO, Zod or Joi, Vitest, Supertest, and Playwright. Keep a modular monolith.

Folders / files: docs/architecture.md; docs/decision-log.md

How to test: Check that the selected stack supports the MVP without requiring microservices.

Acceptance gate: Stack decision is documented and no unnecessary infrastructure is required.

Evidence / notes: _____

☐ P01-02 | Create frontend application shell

What to do: Create the Vite React app, install styling and routing dependencies, and make a basic responsive shell.

Folders / files: frontend/package.json; frontend/src/App.jsx; frontend/src/main.jsx; frontend/src/pages/

How to test: Start the dev server and open the landing page on a desktop and narrow viewport.

Acceptance gate: Frontend starts without console errors and responds to mobile width.

Evidence / notes: _____

☐ P01-03 | Create backend application shell

What to do: Create Express app, route registration, centralized error handling, request logging, and health endpoint.

Folders / files: backend/package.json; backend/src/app.js; backend/src/server.js; backend/src/routes/health.routes.js

How to test: Start backend and request the health endpoint; verify non-health routes return controlled errors.

Acceptance gate: Backend starts and returns a health response.

Evidence / notes: _____

☐ P01-04 | Add configuration and environment validation

What to do: Centralize environment parsing; validate required database URL, origin, cookie settings, and secrets at startup.

Folders / files: backend/src/config/env.js; .env.example; backend/src/config/

How to test: Start with a required variable missing and confirm startup fails clearly; ensure secrets are not committed.

Acceptance gate: Invalid config fails fast and .env.example contains placeholders only.

Evidence / notes: _____

☐ P01-05 | Set up MySQL and Prisma

What to do: Create development database connection, Prisma configuration, first migration workflow, and database access conventions.

Folders / files: backend/prisma/schema.prisma; backend/src/config/database.js; docs/database-design.md

How to test: Run a migration against a clean development database and inspect the created schema.

Acceptance gate: Database connection and migration work from a clean setup.

Evidence / notes: _____

☐ P01-06 | Establish error and response conventions

What to do: Define a consistent API response shape, validation errors, not-found errors, and internal errors without leaking stack traces.

Folders / files: backend/src/middleware/errorHandler.js; backend/src/utils/; docs/api-specification.md

How to test: Trigger invalid input and an intentional server error; check response shapes and logs.

Acceptance gate: Errors are consistent and production responses reveal no secrets.

Evidence / notes: _____

☐ P01-07 | Add linting, formatting, and test scripts

What to do: Configure frontend/backend lint and test commands; add a minimal smoke test.

Folders / files: frontend/package.json; backend/package.json; tests/

How to test: Run all configured checks from a clean checkout.

Acceptance gate: A new contributor can run the documented checks.

Evidence / notes: _____

☐ P01-08 | Document local development setup

What to do: Document prerequisites, environment setup, database migration, start commands, and common setup failures (without embedding secrets).

Folders / files: README.md; docs/development-setup.md

How to test: Follow the guide from a clean checkout without relying on undocumented steps.

Acceptance gate: A developer can start both apps using the documentation.

Evidence / notes: _____

☐ P01-09 | Add basic CI checks

What to do: Run lint, unit tests, and build checks on pull requests using the chosen CI platform.

Folders / files: .github/workflows/ci.yml; package scripts

How to test: Open a test branch and confirm the workflow passes; deliberately break a test and confirm it fails.

Acceptance gate: CI catches a failing check and reports it clearly.

Evidence / notes: _____

PHASE EXIT GATE

☐ Frontend/backend start locally; DB migration works; CI/lint/tests run from a clean checkout.

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P02 | Authentication, users and authorization

Goal: Build trustworthy account handling and server-side access control before adding exam data.

Primary file/folder touchpoints:
backend/src/modules/auth/{auth.routes.js,auth.controller.js,auth.service.js,auth.schema.js}
backend/src/modules/users/
backend/src/middleware/{authenticate.js,authorize.js}
frontend/src/features/auth/
frontend/src/pages/{LoginPage,RegisterPage}.jsx

Checklist: 8 tasks

☐ P02-01 | Create user model and migrations

What to do: Store identity, email, password hash, role, status, institution reference when applicable, and timestamps. Add appropriate uniqueness constraints.

Folders / files: backend/prisma/schema.prisma; backend/prisma/migrations/

How to test: Migrate a clean database and test duplicate identifiers.

Acceptance gate: Schema enforces intended uniqueness and does not store plaintext passwords.

Evidence / notes: _____

☐ P02-02 | Implement registration policy

What to do: Decide whether teacher registration is open, invite-only, or admin-approved; implement that policy rather than trusting a role sent by the client.

Folders / files: backend/src/modules/auth/

How to test: Try registering while submitting an unauthorized role value.

Acceptance gate: Client cannot self-assign ADMIN or another privileged role.

Evidence / notes: _____

☐ P02-03 | Implement password hashing and login

What to do: Use Argon2id for password hashes and generic login failure messages.

Folders / files: backend/src/modules/auth/

How to test: Register, log in with the correct password, and test incorrect credentials.

Acceptance gate: Passwords are hashed; invalid login does not reveal whether an account exists.

Evidence / notes: _____

☐ P02-04 | Implement session lifecycle

What to do: Use short-lived access credentials and a secure session/refresh design; for cookie-based auth, use HttpOnly, Secure in production, suitable SameSite, rotation, and CSRF protection.

Folders / files: backend/src/modules/auth/; backend/src/config/

How to test: Test login, refresh, logout, expired session, and session invalidation.

Acceptance gate: Logout invalidates the session and long-lived secrets are not stored in localStorage.

Evidence / notes: _____

☐ P02-05 | Add authentication middleware

What to do: Protect API routes and establish a consistent authenticated user context.

Folders / files: backend/src/middleware/authenticate.js

How to test: Call protected endpoints without credentials and with expired credentials.

Acceptance gate: Unauthenticated requests are rejected consistently.

Evidence / notes: _____

☐ P02-06 | Add role authorization and ownership checks

What to do: Check role plus resource ownership or membership on every protected operation; do not rely on hidden UI buttons.

Folders / files: backend/src/middleware/authorize.js; relevant module services

How to test: Attempt cross-teacher quiz access and cross-student attempt access by changing IDs.

Acceptance gate: Unauthorized access fails even when a valid ID is known.

Evidence / notes: _____

☐ P02-07 | Build login and registration UI

What to do: Add accessible forms, validation messages, loading states, and error handling.

Folders / files: frontend/src/features/auth/; frontend/src/pages/

How to test: Test keyboard use, invalid input, narrow screen layout, and server errors.

Acceptance gate: Auth screens are usable on Android-sized viewports.

Evidence / notes: _____

☐ P02-08 | Add auth and authorization tests

What to do: Cover login, logout, expired sessions, role boundaries, and resource ownership.

Folders / files: tests/integration/auth.test.js; tests/security/authorization.test.js

How to test: Run tests and confirm forbidden access scenarios fail as expected.

Acceptance gate: Security-critical auth tests pass.

Evidence / notes: _____

PHASE EXIT GATE

☐ Authentication/session tests pass; role and ownership boundaries reject unauthorized requests.

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P03 | Classrooms, enrollment and quiz authoring

Goal: Let teachers manage groups and build a quiz safely.

Primary file/folder touchpoints:
backend/src/modules/classrooms/
backend/src/modules/quizzes/
backend/src/modules/questions/
frontend/src/features/classrooms/
frontend/src/features/quizzes/
frontend/src/pages/teacher/

Checklist: 10 tasks

☐ P03-01 | Create classroom and membership schema

What to do: Model classrooms, institution/teacher ownership, and student memberships with duplicate-enrollment protection.

Folders / files: backend/prisma/schema.prisma; backend/prisma/migrations/

How to test: Create a classroom, enroll a student, and attempt duplicate enrollment.

Acceptance gate: Membership relationships and constraints behave correctly.

Evidence / notes: _____

☐ P03-02 | Implement classroom APIs

What to do: Create, list, view, update, and archive classrooms with ownership checks.

Folders / files: backend/src/modules/classrooms/

How to test: Try accessing a classroom owned by another teacher.

Acceptance gate: Classroom APIs enforce ownership.

Evidence / notes: _____

☐ P03-03 | Add student roster import

What to do: Support validated CSV import with row-level errors and duplicate handling; keep this optional if roster import delays the MVP.

Folders / files: backend/src/modules/classrooms/; docs/roster-import.md

How to test: Import a valid file and one containing duplicate/invalid rows.

Acceptance gate: Valid rows import; errors are actionable; no silent incorrect enrollment.

Evidence / notes: _____

☐ P03-04 | Create quiz and question schema

What to do: Model quiz settings, questions, options, marks, ordering, question type, and answer keys with suitable relationships.

Folders / files: backend/prisma/schema.prisma; docs/database-design.md

How to test: Run migrations and verify foreign keys and ownership relationships.

Acceptance gate: Question and option relationships enforce quiz ownership.

Evidence / notes: _____

☐ P03-05 | Implement quiz draft APIs

What to do: Create, read, update, and archive drafts; validate duration, time window, marks, and allowed settings.

Folders / files: backend/src/modules/quizzes/

How to test: Submit invalid durations, invalid dates, and unauthorized edits.

Acceptance gate: Invalid settings and unauthorized changes are rejected.

Evidence / notes: _____

☐ P03-06 | Implement question authoring APIs

What to do: Add single-answer MCQ first; validate that correct option belongs to the question and that options are well-formed.

Folders / files: backend/src/modules/questions/

How to test: Try attaching an option from a different question and malformed MCQ data.

Acceptance gate: Question data cannot reference unrelated options.

Evidence / notes: _____

☐ P03-07 | Build quiz editor UI

What to do: Add title, description, timing, marks, question editing, ordering, and draft save.

Folders / files: frontend/src/features/quizzes/; frontend/src/features/questions/

How to test: Create and edit a quiz on desktop and Android-sized viewport.

Acceptance gate: Teacher can save and reopen a draft.

Evidence / notes: _____

☐ P03-08 | Build quiz preview and publish workflow

What to do: Preview student-facing content and publish through a server-controlled state transition.

Folders / files: frontend/src/pages/teacher/; backend/src/modules/quizzes/

How to test: Attempt to publish invalid quiz and attempt to edit locked settings after start.

Acceptance gate: Only valid quizzes publish; state rules are enforced.

Evidence / notes: _____

☐ P03-09 | Protect answer keys

What to do: Create student question DTOs that omit correct answers and grading-only fields.

Folders / files: backend/src/modules/questions/; backend/src/modules/attempts/

How to test: Inspect API response and browser network payload for answer-key leakage.

Acceptance gate: No correct answer is exposed before grading.

Evidence / notes: _____

☐ P03-10 | Test quiz and question permissions

What to do: Add tests for ownership, invalid question relations, and draft/published state rules.

Folders / files: tests/integration/quizzes.test.js; tests/security/

How to test: Run the quiz module test suite.

Acceptance gate: Quiz authoring and authorization tests pass.

Evidence / notes: _____

PHASE EXIT GATE

☐ **Teacher creates and publishes a quiz; question APIs never expose answer keys.**

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P04 | Invitations, exam sessions and student entry

Goal: Create a controlled path from a teacher's published quiz to an eligible student attempt.

Primary file/folder touchpoints:
backend/src/modules/invitations/
backend/src/modules/examSessions/
backend/src/modules/participants/
frontend/src/pages/join/
frontend/src/features/exam-entry/

Checklist: 8 tasks

☐ P04-01 | Define exam-session schema and state machine

What to do: Separate reusable quiz definition from a particular scheduled/running exam session; define legal transitions and transition actors.

Folders / files: backend/prisma/schema.prisma; docs/exam-state-machine.md

How to test: Test valid and invalid transitions in a state-transition table.

Acceptance gate: Every transition has a documented rule.

Evidence / notes: _____

☐ P04-02 | Create secure invitation tokens

What to do: Generate cryptographically random, high-entropy tokens; store a token hash when practical; support expiry and revocation.

Folders / files: backend/src/modules/invitations/

How to test: Check token uniqueness, expiry, revocation, and that raw tokens do not appear in logs.

Acceptance gate: Invitation tokens are unpredictable and revocable.

Evidence / notes: _____

☐ P04-03 | Implement invitation creation and revocation

What to do: Allow only the quiz owner/authorized teacher to create, view, rotate, and revoke invitations.

Folders / files: backend/src/modules/invitations/

How to test: Try creating/revoking another teacher's invitation.

Acceptance gate: Ownership and state checks are enforced.

Evidence / notes: _____

☐ P04-04 | Implement student identity and eligibility checks

What to do: Validate enrollment, account status, invitation validity, exam availability, and attempt limits.

Folders / files: backend/src/modules/participants/; backend/src/modules/attempts/

How to test: Try joining with expired link, un-enrolled student, and excess attempt.

Acceptance gate: Only eligible students can join.

Evidence / notes: _____

☐ P04-05 | Build join page

What to do: Provide a clear mobile-first join page and collect only required identity fields; do not treat typed data as proof of identity.

Folders / files: frontend/src/pages/join/; frontend/src/features/exam-entry/

How to test: Open link on Android and test invalid, expired, and valid invitations.

Acceptance gate: Student receives clear and correct join feedback.

Evidence / notes: _____

☐ P04-06 | Build waiting room

What to do: Show exam details, rules, readiness checks, and waiting status without exposing questions early.

Folders / files: frontend/src/features/exam-entry/; backend/src/modules/examSessions/

How to test: Verify students cannot load exam questions before the session permits it.

Acceptance gate: Waiting-room state is enforced by the server.

Evidence / notes: _____

☐ P04-07 | Implement server-controlled start/pause/end

What to do: Allow only authorized teacher actions and enforce legal state transitions atomically.

Folders / files: backend/src/modules/examSessions/

How to test: Send duplicate start requests and attempt illegal transitions.

Acceptance gate: State transitions remain consistent under repeated requests.

Evidence / notes: _____

☐ P04-08 | Test invitation and join flows

What to do: Cover token expiry, replay, revocation, eligibility, and simultaneous joins.

Folders / files: tests/integration/invitations.test.js; tests/e2e/join-flow.spec.js

How to test: Run the integration and browser tests.

Acceptance gate: Entry workflow passes all defined cases.

Evidence / notes: _____

PHASE EXIT GATE

☐ Valid enrolled student joins through invitation; invalid/expired/revoked link is rejected.

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P05 | Student examination engine and answer integrity

Goal: Deliver a reliable quiz-taking experience with server-authoritative timing and durable answers.

Primary file/folder touchpoints:
backend/src/modules/attempts/
backend/src/modules/answers/
backend/src/modules/submissions/
frontend/src/features/examination/
frontend/src/pages/student/ExamPage.jsx

Checklist: 11 tasks

☐ P05-01 | Create attempt model and transition rules

What to do: Track participant, attempt number, status, start, server deadline, last activity, and finalization.

Folders / files: backend/prisma/schema.prisma; docs/attempt-state-machine.md

How to test: Test every allowed and disallowed attempt transition.

Acceptance gate: Attempts cannot jump into invalid states.

Evidence / notes: _____

☐ P05-02 | Create question delivery endpoint

What to do: Return only permitted questions/options for the authenticated active attempt; avoid exposing answer keys or unrelated questions.

Folders / files: backend/src/modules/attempts/; backend/src/modules/questions/

How to test: Try requesting another attempt or another quiz's question.

Acceptance gate: Question delivery is scoped to the attempt.

Evidence / notes: _____

☐ P05-03 | Implement answer validation and saving

What to do: Validate attempt ownership, active state, question membership, answer shape, deadline, and concurrency/version rules.

Folders / files: backend/src/modules/answers/

How to test: Submit valid/invalid answers, stale versions, and answers for unrelated questions.

Acceptance gate: Only valid current answers are saved.

Evidence / notes: _____

☐ P05-04 | Implement idempotent answer requests

What to do: Use request IDs or versions so retrying a request does not create duplicate side effects or overwrite newer answers.

Folders / files: backend/src/modules/answers/

How to test: Replay the same request and send requests out of order.

Acceptance gate: Duplicate/stale requests are handled predictably.

Evidence / notes: _____

☐ P05-05 | Build mobile-first exam interface

What to do: Create question view, navigation, answer controls, countdown, progress, save state, and submit confirmation.

Folders / files: frontend/src/features/examination/; frontend/src/pages/student/

How to test: Test on a real Android browser and desktop viewport; verify keyboard/touch accessibility.

Acceptance gate: Core exam UI is usable and responsive.

Evidence / notes: _____

☐ P05-06 | Implement server-authoritative timer

What to do: Calculate deadlines on the server; client countdown is display-only; define pause/extension policy explicitly.

Folders / files: backend/src/modules/attempts/; frontend/src/features/examination/

How to test: Change device clock and client countdown; try late answer submission.

Acceptance gate: Server deadline remains authoritative.

Evidence / notes: _____

☐ P05-07 | Implement autosave acknowledgement UI

What to do: Show saving/saved/failed/reconnecting states only based on server acknowledgement.

Folders / files: frontend/src/features/examination/; backend/src/modules/answers/

How to test: Throttle/offline the network and confirm the UI does not falsely claim saved.

Acceptance gate: Displayed save state matches backend acknowledgement.

Evidence / notes: _____

☐ P05-08 | Add local draft recovery carefully

What to do: Use IndexedDB only as a recovery aid; reconcile local pending changes with server state and version conflicts.

Folders / files: frontend/src/features/examination/; docs/recovery-policy.md

How to test: Refresh or disconnect during an answer change and restore the attempt.

Acceptance gate: Recovery does not overwrite newer server data or bypass deadline.

Evidence / notes: _____

☐ P05-09 | Implement submission and finalization

What to do: Validate deadline/state, finalize answers and submission transactionally, and make repeated submission safe.

Folders / files: backend/src/modules/submissions/; backend/src/modules/attempts/

How to test: Submit twice concurrently and after deadline; inspect final database state.

Acceptance gate: Exactly one final outcome is produced.

Evidence / notes: _____

☐ P05-10 | Implement automatic MCQ grading

What to do: Grade on the server, apply marks/negative marks, and separate grading from student-supplied data.

Folders / files: backend/src/modules/results/; backend/src/modules/submissions/

How to test: Test correct, incorrect, unanswered, and malformed answers.

Acceptance gate: Scores are computed only from trusted answer keys.

Evidence / notes: _____

☐ P05-11 | Test exam failure and recovery scenarios

What to do: Test refresh, offline/online, reconnect, duplicate tabs, duplicate submit, server restart, and late answers.

Folders / files: tests/integration/answers.test.js; tests/e2e/exam-flow.spec.js

How to test: Run scenario checklist and inspect saved answers and final state.

Acceptance gate: No critical data-loss or state-corruption issue remains.

Evidence / notes: _____

PHASE EXIT GATE

☐ Student can finish an exam; server validates deadline/submission; autosave/recovery tests pass.

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P06 | Browser telemetry, security events and risk assessment

Goal: Collect supported signals without overstating what a browser can prove.

Primary file/folder touchpoints:
backend/src/modules/security/
backend/src/modules/riskAssessment/
frontend/src/features/security-monitoring/
frontend/src/hooks/useVisibilityEvents.js
docs/security-event-catalog.md
docs/security-policy.md

Checklist: 11 tasks

☐ P06-01 | Define event catalog and semantics

What to do: Define names, payload fields, meaning, limitations, retention, and severity for page visibility, focus, fullscreen, clipboard, and connection events.

Folders / files: docs/security-event-catalog.md

How to test: Review each event and confirm the name does not claim more than the browser observed.

Acceptance gate: Each event has a precise and non-misleading definition.

Evidence / notes: _____

☐ P06-02 | Implement page visibility event collection

What to do: Record supported visibility changes during active attempts and avoid duplicate listeners across rerenders.

Folders / files: frontend/src/features/security-monitoring/; frontend/src/hooks/

How to test: Switch tabs, minimize, return, and inspect recorded events on supported browsers.

Acceptance gate: Expected visibility events are collected without duplicate handlers.

Evidence / notes: _____

☐ P06-03 | Implement focus/fullscreen event collection

What to do: Observe supported focus and fullscreen changes; request fullscreen only after user gesture and handle unsupported devices.

Folders / files: frontend/src/features/security-monitoring/

How to test: Test enter/exit fullscreen and focus changes on each supported browser.

Acceptance gate: Events and fallback UI match actual browser capabilities.

Evidence / notes: _____

☐ P06-04 | Implement clipboard restrictions with accessibility review

What to do: Block supported copy/cut/paste interactions only within the exam surface where policy requires; do not promise complete prevention.

Folders / files: frontend/src/features/security-monitoring/

How to test: Test mouse, keyboard, touch, and assistive workflows; confirm restrictions do not break normal answer input.

Acceptance gate: Policy works as intended without avoidable accessibility breakage.

Evidence / notes: _____

☐ P06-05 | Implement security-event API validation

What to do: Authenticate attempt, validate event type/payload size/frequency, assign server receipt time, and ignore client-supplied score.

Folders / files: backend/src/modules/security/

How to test: Send unsupported events, oversized payloads, and fabricated score fields.

Acceptance gate: Invalid events are rejected or safely ignored.

Evidence / notes: _____

☐ P06-06 | Add event deduplication and rate limits

What to do: Use client event UUID plus database uniqueness; cap event rates and per-attempt volume.

Folders / files: backend/src/modules/security/; backend/prisma/schema.prisma

How to test: Replay event IDs and flood events within test limits.

Acceptance gate: Duplicate events do not increase risk repeatedly; floods are controlled.

Evidence / notes: _____

☐ P06-07 | Persist immutable-style event history

What to do: Store event, attempt, client ID, server receipt time, policy version, and limited metadata; restrict updates/deletes.

Folders / files: backend/src/modules/security/; backend/prisma/schema.prisma

How to test: Verify history can be queried and normal app paths cannot rewrite old events.

Acceptance gate: Event history supports later review and is access-controlled.

Evidence / notes: _____

☐ P06-08 | Define initial scoring policy

What to do: Use configurable, illustrative rule-based points; keep network health and presence status separate from security risk; document false-positive limitations.

Folders / files: backend/src/modules/riskAssessment/; docs/security-policy.md

How to test: Unit-test score changes, caps, repeated events, and policy versions.

Acceptance gate: Risk score is computed server-side and explainable.

Evidence / notes: _____

☐ P06-09 | Implement risk assessment and explanation

What to do: Store/recompute assessment from accepted events and policy version; expose contributing events and reason labels.

Folders / files: backend/src/modules/riskAssessment/

How to test: Compare score to a manually calculated expected result.

Acceptance gate: Every displayed score can be explained from its contributing events.

Evidence / notes: _____

☐ P06-10 | Build student warnings

What to do: Show understandable warning when policy triggers; do not claim that an event proves cheating.

Folders / files: frontend/src/features/security-monitoring/

How to test: Trigger a test event and verify correct message and recovery path.

Acceptance gate: Warnings are accurate, accessible, and policy-based.

Evidence / notes: _____

☐ P06-11 | Test spoofing and false positives

What to do: Try fabricated events, duplicate events, client score manipulation, accidental visibility changes, and unsupported APIs.

Folders / files: tests/security/security-events.test.js; docs/test-strategy.md

How to test: Run adversarial and ordinary-use scenarios.

Acceptance gate: Server remains authoritative; false-positive behaviour is documented.

Evidence / notes: _____

PHASE EXIT GATE

☐ Supported events persist, duplicate events are deduplicated, and risk score is server-computed/explainable.

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P07 | Live monitoring and teacher incident workflow

Goal: Deliver live participant state, event alerts, timelines, and auditable teacher actions.

Primary file/folder touchpoints:
backend/src/sockets/
backend/src/modules/monitoring/
backend/src/modules/teacherActions/
frontend/src/features/monitoring/
frontend/src/pages/teacher/LiveExamPage.jsx

Checklist: 9 tasks

☐ P07-01 | Define real-time event contract

What to do: Document socket events and payloads for participant join, progress, security updates, connection state, and submission.

Folders / files: docs/realtime-events.md

How to test: Review payloads to ensure they expose only necessary information.

Acceptance gate: Each event has a defined producer, consumer, and authorization rule.

Evidence / notes: _____

☐ P07-02 | Authenticate Socket.IO connections

What to do: Authenticate sockets and validate role, attempt participation, and exam ownership before joining rooms.

Folders / files: backend/src/sockets/

How to test: Try joining another teacher's room or another student's attempt room.

Acceptance gate: Unauthorized room joins are rejected.

Evidence / notes: _____

☐ P07-03 | Persist before publishing security updates

What to do: Save accepted event and assessment before publishing the update; document failure handling and later outbox option.

Folders / files: backend/src/modules/security/; backend/src/sockets/

How to test: Simulate database failure and verify no false persisted-looking alert is broadcast.

Acceptance gate: Dashboard updates correspond to accepted persisted state.

Evidence / notes: _____

☐ P07-04 | Track connection health separately

What to do: Track connected/reconnecting/disconnected and last-seen; use grace periods so a transient drop is not treated as cheating.

Folders / files: backend/src/sockets/; backend/src/modules/monitoring/

How to test: Disconnect a client temporarily and reconnect within/after grace period.

Acceptance gate: Connection status is accurate and does not add security points by default.

Evidence / notes: _____

☐ P07-05 | Build live overview dashboard

What to do: Show participant counts, attempt status, progress, presence status, connection health, and risk levels.

Folders / files: frontend/src/features/monitoring/; frontend/src/pages/teacher/LiveExamPage.jsx

How to test: Open teacher and student browsers and verify updates without refresh.

Acceptance gate: Live dashboard updates correctly and shows no unauthorized data.

Evidence / notes: _____

☐ P07-06 | Build filters and student detail drawer/page

What to do: Filter by risk, connection, presence, status; show student details and incident timeline.

Folders / files: frontend/src/features/monitoring/

How to test: Filter students and inspect a single student's history.

Acceptance gate: Teacher can quickly locate and understand an incident.

Evidence / notes: _____

☐ P07-07 | Implement warning and termination actions

What to do: Require authorization, confirmation, permitted state, reason where appropriate, and audit record.

Folders / files: backend/src/modules/teacherActions/; frontend/src/features/monitoring/

How to test: Attempt action as student, on another teacher's exam, and twice.

Acceptance gate: Only permitted actions succeed and are auditable.

Evidence / notes: _____

☐ P07-08 | Handle refresh and socket reconnect

What to do: Reload dashboard from REST snapshot, then subscribe to live updates; avoid relying on socket history alone.

Folders / files: frontend/src/features/monitoring/; backend/src/modules/monitoring/

How to test: Refresh dashboard during an exam and reconnect socket.

Acceptance gate: Dashboard reconstructs correct current state.

Evidence / notes: _____

☐ P07-09 | Test real-time load and isolation

What to do: Simulate multiple participants and ensure each teacher/student receives only authorized updates.

Folders / files: tests/integration/monitoring.test.js; tests/e2e/monitoring.spec.js

How to test: Run a small load test and cross-room authorization tests.

Acceptance gate: No cross-exam data leak; updates remain usable at target demo scale.

Evidence / notes: _____

PHASE EXIT GATE

☐ Teacher sees authorized live updates, can review timeline, and actions are audited.

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P08 | Classroom presence verification

Goal: Implement QR-based classroom challenges first and preserve clear limitations of every presence signal.

Primary file/folder touchpoints:
backend/src/modules/presence/
backend/src/modules/classrooms/
frontend/src/features/presence/
frontend/src/pages/teacher/PresencePanel.jsx
docs/presence-model.md

Checklist: 7 tasks

☐ P08-01 | Define presence statuses and claims

What to do: Use explicit states such as UNVERIFIED, VERIFIED_BY_QR, VERIFIED_BY_TEACHER, and EXCEPTION_REVIEW_REQUIRED; do not equate one signal with proof of location.

Folders / files: docs/presence-model.md; backend/src/modules/presence/

How to test: Review UI wording and report output for overclaiming.

Acceptance gate: Presence status describes evidence, not certainty beyond it.

Evidence / notes: _____

☐ P08-02 | Create short-lived challenge model

What to do: Bind challenge to exam session/classroom, expiry, creator, status, and single-use/replay policy as appropriate.

Folders / files: backend/prisma/schema.prisma; backend/src/modules/presence/

How to test: Test expired, revoked, and reused challenges.

Acceptance gate: Challenge lifetime and reuse rules are enforced.

Evidence / notes: _____

☐ P08-03 | Generate QR challenge

What to do: Generate unpredictable challenge token and QR URL on the teacher side through the backend; avoid permanent classroom QR codes.

Folders / files: backend/src/modules/presence/; frontend/src/features/presence/

How to test: Generate, rotate, expire, and revoke QR challenges.

Acceptance gate: Only valid current challenge can be accepted.

Evidence / notes: _____

☐ P08-04 | Implement student verification

What to do: Validate authenticated participant, active session, challenge, expiry, replay, and rate limits; save verification outcome.

Folders / files: backend/src/modules/presence/; frontend/src/features/presence/

How to test: Test eligible student, un-enrolled student, expired token, and replay.

Acceptance gate: Presence verification is server-controlled.

Evidence / notes: _____

☐ P08-05 | Build teacher presence panel

What to do: Display which participants completed each method and allow manual confirmation/exception review with audit trail.

Folders / files: frontend/src/pages/teacher/PresencePanel.jsx; backend/src/modules/teacherActions/

How to test: Verify live status updates and manual action audit entries.

Acceptance gate: Teacher can distinguish verified, unverified, and exception states.

Evidence / notes: _____

☐ P08-06 | Document network/BLE future integration

What to do: Explain that browser APIs do not reliably expose Wi-Fi SSID or universal BLE scanning; outline institution network integration and optional native-app BLE later.

Folders / files: docs/presence-model.md; docs/roadmap.md

How to test: Review technical assumptions before implementation is scheduled.

Acceptance gate: No unsupported browser capability is treated as available.

Evidence / notes: _____

☐ P08-07 | Test presence abuse and accessibility

What to do: Test QR sharing/replay limits, mobile-data users, failed camera scanning, and manual fallback.

Folders / files: tests/security/presence.test.js; tests/e2e/presence.spec.js

How to test: Run cases on real Android devices and desktop fallback.

Acceptance gate: Failure paths are clear and legitimate students have a defined fallback.

Evidence / notes: _____

PHASE EXIT GATE

☐ Short-lived QR verification works; expiry/replay/eligibility and manual fallback are tested.

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P09 | Results, audit records and reporting

Goal: Finish the teacher workflow and create useful, traceable examination records.

Primary file/folder touchpoints:
backend/src/modules/results/
backend/src/modules/audit/
backend/src/modules/reports/
frontend/src/features/results/
frontend/src/pages/teacher/ResultsPage.jsx
docs/retention-policy.md

Checklist: 7 tasks

☐ P09-01 | Finalize grading and result schema

What to do: Separate submission from result where useful; define score, grading status, publication state, and manual grading fields.

Folders / files: backend/prisma/schema.prisma; backend/src/modules/results/

How to test: Test score calculation and result state transitions.

Acceptance gate: Result states and score fields are clearly defined.

Evidence / notes: _____

☐ P09-02 | Implement result access controls

What to do: Students see only their own results; teachers see only authorized exam results; admin access is explicit.

Folders / files: backend/src/modules/results/

How to test: Try reading another student's result and another teacher's result set.

Acceptance gate: No unauthorized result access succeeds.

Evidence / notes: _____

☐ P09-03 | Build results dashboard

What to do: Show scores, submission status, and relevant exam summary; respect result-publication policy.

Folders / files: frontend/src/features/results/; frontend/src/pages/teacher/ResultsPage.jsx

How to test: Test before and after results are published.

Acceptance gate: Students cannot see results prematurely.

Evidence / notes: _____

☐ P09-04 | Add CSV export

What to do: Export authorized results with safe column names and avoid spreadsheet formula injection in user-controlled cells.

Folders / files: backend/src/modules/reports/; frontend/src/features/results/

How to test: Export a sample class and test cells beginning with formula characters.

Acceptance gate: Export is accurate and safe to open in common spreadsheet tools.

Evidence / notes: _____

☐ P09-05 | Implement audit records

What to do: Record important teacher/admin actions, state transitions, grading changes, and publication events with actor, timestamp, target, and reason where appropriate.

Folders / files: backend/src/modules/audit/; docs/audit-model.md

How to test: Perform actions and verify actor/target/time records; try access as an unauthorized user.

Acceptance gate: Audit trail is complete for defined high-value actions.

Evidence / notes: _____

☐ P09-06 | Define privacy and retention rules

What to do: Specify collected data, access, retention, deletion, export, and institutional responsibilities; minimize unnecessary personal/device data.

Folders / files: docs/retention-policy.md; docs/security-model.md

How to test: Review every database field and report against data minimization rules.

Acceptance gate: Retention and privacy decisions are documented.

Evidence / notes: _____

☐ P09-07 | Create incident report

What to do: Generate a readable report showing accepted events, policy version, risk explanation, connection state, presence outcome, teacher actions, and final result.

Folders / files: backend/src/modules/reports/; frontend/src/features/monitoring/

How to test: Compare report to event history and ensure interpretations are labelled as interpretations.

Acceptance gate: Report is traceable and does not claim that risk equals proven misconduct.

Evidence / notes: _____

PHASE EXIT GATE

☐ **Results are access-controlled; exports and audit/incident reports are accurate.**

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P10 | Security review, reliability, deployment and release

Goal: Prove the main workflows work, harden the app, deploy it, and record the release baseline.

Primary file/folder touchpoints:
tests/unit/
tests/integration/
tests/e2e/
tests/security/
docs/deployment.md
docs/release-checklist.md
deployment/ (if used)
README.md

Checklist: 10 tasks

☐ P10-01 | Complete unit and integration test coverage for critical rules

What to do: Prioritize authorization, state transitions, answer validation, deadline enforcement, event deduplication, risk scoring, presence tokens, and grading.

Folders / files: tests/unit/; tests/integration/

How to test: Run test suite and inspect failures rather than skipping them.

Acceptance gate: Critical business/security rules have repeatable tests.

Evidence / notes: _____

☐ P10-02 | Run end-to-end exam scenario

What to do: Automate or document teacher creates exam, student joins, answers, event arrives, teacher reviews, student submits, result appears.

Folders / files: tests/e2e/

How to test: Run the full scenario from a clean database.

Acceptance gate: Full core workflow passes repeatedly.

Evidence / notes: _____

☐ P10-03 | Perform adversarial authorization review

What to do: Test object-ID tampering, role escalation, answer-key leakage, invitation replay, event spoofing, and socket room access.

Folders / files: tests/security/; docs/threat-model.md

How to test: Record each test, outcome, and remediation.

Acceptance gate: No known critical authorization/data leakage issue remains open.

Evidence / notes: _____

☐ P10-04 | Test Android browser compatibility

What to do: Test supported Android browsers on real devices; record limitations for fullscreen, visibility, clipboard, QR scanning, and reconnect.

Folders / files: docs/browser-compatibility.md; tests/e2e/

How to test: Run the same checklist on each supported browser/device combination.

Acceptance gate: Supported-browser matrix reflects actual test results.

Evidence / notes: _____

☐ P10-05 | Test reconnect, backup and recovery

What to do: Test app restart, database backup/restore process, answer recovery, and socket reconnect; verify the provider's backup features.

Folders / files: docs/operations-runbook.md; docs/deployment.md

How to test: Perform a documented restore rehearsal in a non-production environment.

Acceptance gate: Recovery procedure is documented and has been tested.

Evidence / notes: _____

☐ P10-06 | Configure production environment

What to do: Set production origins, secrets, database credentials, secure cookies, HTTPS, logging, health checks, and restrictive CORS; do not commit secrets.

Folders / files: backend/src/config/; deployment/; docs/deployment.md

How to test: Review production configuration and inspect logs for secret leakage.

Acceptance gate: Production configuration is secure and reproducible.

Evidence / notes: _____

☐ P10-07 | Deploy frontend, backend, and managed MySQL

What to do: Deploy to the selected host/provider; verify migrations, environment variables, custom domain if used, and HTTPS.

Folders / files: docs/deployment.md; deployment/

How to test: Run health checks and a small real end-to-end exam after deployment.

Acceptance gate: Production demo is reachable and core workflow succeeds.

Evidence / notes: _____

☐ P10-08 | Add operational monitoring and safe logs

What to do: Log request IDs and errors without passwords, tokens, answer keys, or unnecessary student data; add health/status checks.

Folders / files: backend/src/middleware/; backend/src/config/; docs/operations-runbook.md

How to test: Trigger controlled errors and inspect logs for usefulness and sensitive-data absence.

Acceptance gate: Logs support debugging without exposing secrets.

Evidence / notes: _____

☐ P10-09 | Create release checklist and known-limitations page

What to do: Record supported features, browser limitations, open issues, backup status, and how to report problems.

Folders / files: docs/release-checklist.md; docs/known-limitations.md; README.md

How to test: Ask a reviewer to follow the checklist and find the limitation statements.

Acceptance gate: Release scope and limitations are clear.

Evidence / notes: _____

☐ P10-10 | Record MVP baseline and next backlog

What to do: Tag the release, record test results, note known defects, and prioritize next release features.

Folders / files: docs/decision-log.md; docs/roadmap.md; Git release tag

How to test: Verify tag points to the tested commit and backlog has priorities.

Acceptance gate: MVP can be reproduced and future work is prioritized.

Evidence / notes: _____

PHASE EXIT GATE

☐ **Critical tests pass; Android compatibility is recorded; deployment and recovery are rehearsed.**

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

P11 | Post-MVP enhancements and scale-up

Goal: Keep later features organized without letting them block the first reliable release.

Primary file/folder touchpoints:
docs/roadmap.md
docs/feature-backlog.md
backend/src/modules/ (new modules only when justified)
frontend/src/features/ (new features only when justified)

Checklist: 10 tasks

☐ P11-01 | Add question pools and controlled randomization

What to do: Select balanced question subsets by topic/difficulty and record the exam version used for each attempt.

Folders / files: backend/src/modules/questions/; backend/src/modules/examSessions/

How to test: Verify each attempt gets a valid set and grading maps to the correct questions.

Acceptance gate: Randomization does not break grading or auditability.

Evidence / notes: _____

☐ P11-02 | Add additional question types

What to do: Add multiple-select, true/false, numerical, short-answer, and essay/manual grading incrementally.

Folders / files: backend/src/modules/questions/; backend/src/modules/results/; frontend/src/features/examination/

How to test: Create tests for each type and its grading rules.

Acceptance gate: Each question type has validated input and documented grading.

Evidence / notes: _____

☐ P11-03 | Improve institutional identity integration

What to do: Consider institutional SSO or verified email where appropriate and available.

Folders / files: backend/src/modules/auth/; docs/architecture.md

How to test: Test identity mapping, logout, and role provisioning in a test institution.

Acceptance gate: Institution integration does not weaken authorization.

Evidence / notes: _____

☐ P11-04 | Investigate network-side presence integration

What to do: Only proceed with network administrator cooperation; define exactly what the signal proves and its failure modes.

Folders / files: backend/src/modules/presence/; docs/presence-model.md

How to test: Test with the actual managed network and a documented threat model.

Acceptance gate: Network signal is not presented as proof beyond its capabilities.

Evidence / notes: _____

☐ P11-05 | Investigate BLE via supported native app

What to do: Only if needed; assess platform permissions, relay/spoofing risk, battery impact, and app distribution first.

Folders / files: docs/presence-model.md; future mobile app repository/module

How to test: Prototype on target Android devices before committing to product requirements.

Acceptance gate: BLE is an optional signal, not sole proof of presence.

Evidence / notes: _____

☐ P11-06 | Investigate dedicated examination environment integration

What to do: Evaluate existing tools such as Safe Exam Browser before writing device-lockdown software from scratch.

Folders / files: docs/architecture.md; docs/decision-log.md

How to test: Create a compatibility proof-of-concept and document supported platforms.

Acceptance gate: Integration is justified by a concrete requirement.

Evidence / notes: _____

☐ P11-07 | Add analytics and question quality reports

What to do: Analyse difficulty, completion time, score distribution, unanswered questions, and common incorrect options.

Folders / files: backend/src/modules/reports/; frontend/src/features/results/

How to test: Validate calculations on a known sample dataset.

Acceptance gate: Analytics are correct and do not expose unauthorized student data.

Evidence / notes: _____

☐ P11-08 | Scale real-time infrastructure only when needed

What to do: If multiple backend instances are required, plan shared Socket.IO adapter, shared rate limits, background jobs, and an outbox strategy.

Folders / files: backend/src/sockets/; backend/src/jobs/; docs/architecture.md

How to test: Load-test target participant count and test cross-instance updates.

Acceptance gate: Scaling change addresses measured bottlenecks rather than adding complexity prematurely.

Evidence / notes: _____

☐ P11-09 | Evaluate privacy-sensitive proctoring separately

What to do: Camera/microphone features require explicit purpose, consent, minimization, retention, security review, and alternatives; not part of core MVP.

Folders / files: docs/privacy-impact-assessment.md; docs/feature-backlog.md

How to test: Complete privacy/security review before any implementation.

Acceptance gate: No recording feature is enabled without a documented approval process.

Evidence / notes: _____

☐ P11-10 | Plan dedicated mobile app only after browser MVP

What to do: Reuse backend APIs and domain rules; document native-only capabilities and separate release/support burden.

Folders / files: docs/roadmap.md; future mobile app docs

How to test: Approve a product case and technical proof-of-concept before creating a separate app.

Acceptance gate: Mobile app does not fork the core exam rules.

Evidence / notes: _____

PHASE EXIT GATE

☐ **Enhancement has a documented product need, tests, privacy/security review where applicable, and no MVP regression.**

Gate review date: _____ Commit / test evidence: _____ Reviewed by: _____

9. Master verification checklist

Use the Test Plan worksheet to record status, observed result, evidence, and any issue. These scenarios should be run as the relevant modules are built, then rerun as regression tests before release.

☐ T-001 | Authentication — Unauthenticated request to protected API

Expected result: Rejected without sensitive data.

Observed result / evidence: _____

☐ T-002 | Authorization — Student changes attempt ID to another student's attempt

Expected result: Access denied; no data leaked.

Observed result / evidence: _____

☐ T-003 | Authorization — Teacher changes quiz ID to another teacher's quiz

Expected result: Access denied.

Observed result / evidence: _____

☐ T-004 | Answer security — Inspect student question API payload

Expected result: Correct answers and grading-only fields absent.

Observed result / evidence: _____

☐ T-005 | Invitation — Use expired or revoked invitation

Expected result: Join rejected with clear message.

Observed result / evidence: _____

☐ T-006 | Eligibility — Un-enrolled student attempts to join restricted exam

Expected result: Join rejected.

Observed result / evidence: _____

☐ T-007 | Timer — Change local device clock

Expected result: Server deadline remains authoritative.

Observed result / evidence: _____

☐ T-008 | Submission — Submit twice concurrently

Expected result: One final outcome; duplicate handled safely.

Observed result / evidence: _____

☐ T-009 | Answer integrity — Submit an option from a different question

Expected result: Rejected.

Observed result / evidence: _____

☐ T-010 | Autosave — Drop network during answer save

Expected result: UI shows failure/reconnect, not falsely saved.

Observed result / evidence: _____

☐ T-011 | Recovery — Refresh/reconnect after pending answer

Expected result: State reconciles without overwriting newer server data.

Observed result / evidence: _____

☐ T-012 | Events — Replay same event UUID

Expected result: Event is deduplicated and score not inflated.

Observed result / evidence: _____

☐ T-013 | Events — Submit client-supplied risk score

Expected result: Ignored/rejected; backend calculates score.

Observed result / evidence: _____

☐ T-014 | Events — Flood event endpoint

Expected result: Rate limits/volume limits apply.

Observed result / evidence: _____

☐ T-015 | Monitoring — Student tries to join teacher socket room

Expected result: Rejected.

Observed result / evidence: _____

☐ T-016 | Monitoring — Teacher refreshes live dashboard

Expected result: REST snapshot restores current state; socket resumes.

Observed result / evidence: _____

☐ T-017 | Presence — Replay expired QR challenge

Expected result: Rejected.

Observed result / evidence: _____

☐ T-018 | Presence — Student on mobile data uses valid QR

Expected result: Handled according to policy; mobile data alone is not a failure.

Observed result / evidence: _____

☐ T-019 | Privacy — Inspect logs for passwords/tokens/answer keys

Expected result: No sensitive secrets are logged.

Observed result / evidence: _____

☐ T-020 | Reporting — CSV field starts with spreadsheet formula character

Expected result: Export safely neutralizes formula injection.

Observed result / evidence: _____

☐ T-021 | Browser compatibility — Fullscreen unsupported on a target browser

Expected result: Clear fallback/eligibility behaviour; no false pass.

Observed result / evidence: _____

☐ T-022 | End-to-end — Teacher creates exam through published result

Expected result: Full flow passes on supported target browser.

Observed result / evidence: _____

☐ T-023 | Reliability — Backend restarts during active attempt

Expected result: Attempt/answers recover according to documented policy.

Observed result / evidence: _____

☐ T-024 | Audit — Teacher terminates an attempt

Expected result: Actor, target, time, and reason are recorded.

Observed result / evidence: _____

10. Deployment and MVP release gate

Functional readiness

☐ Teacher can create, publish, start, monitor, and end an exam.

☐ An eligible student can join on a supported Android browser, answer, and submit.

☐ Correct answers remain server-side until grading.

☐ Autosave, deadline enforcement, and duplicate submission behaviour are tested.

☐ Results are calculated and displayed only to authorized users.

Security readiness

☐ Critical authorization/object-ID tests pass.

☐ Invitation and QR expiry/replay checks pass.

☐ Security events are validated, deduplicated, and server-scored.

☐ Socket rooms enforce authorization.

☐ Logs do not contain passwords, secrets, answer keys, or unnecessary student data.

Reliability readiness

☐ Reconnect and answer recovery have been tested.

☐ Database backup and restore procedure is documented and rehearsed.

☐ Supported Android browser matrix reflects real-device tests.

☐ Health checks and operational error handling are configured.

☐ Known limitations and failure behaviour are documented.

Release evidence

☐ Critical tests have recorded outcomes.

☐ No known critical security or data-loss issue remains open.

☐ Release commit/tag identifies the tested version.

☐ Deployment steps and rollback/recovery procedure are documented.

☐ MVP scope and deferred features are clearly recorded.

Release record

Field	Record
Release version	_____
Release date	_____
Commit / tag	_____
Critical tests passed?	Yes / No / Not yet
Open blockers	_____
Known limitations reviewed?	Yes / No
Final decision	GO / NO-GO / REQUIRES FOLLOW-UP

11. Post-MVP feature roadmap

These features are intentionally separate from the initial release. Prioritize them only after the core examination flow and reliability gates pass.

Priority	Feature	When to consider	Main validation needed
P1	Question pools and balanced random selection	Release 2	Question/version mapping and grading remain correct for every attempt.
P1	More question types: multi-select, true/false, numerical, short answer, essay	Release 2	Input validation and grading rules per type.
P1	Institutional SSO	When institution can provide identity integration	Identity mapping, provisioning, logout, role boundaries.
P1	Network-side presence integration	Only with institution network administrator	Document precisely what the signal proves and its failure modes.
P2	BLE through supported native application	Only if a real use case requires it	Prototype device support, permissions, relay/spoofing risks, battery impact.
P2	Dedicated examination environment integration	After browser MVP	Evaluate existing tools, such as Safe Exam Browser, before recreating lockdown.
P2	Question-quality analytics	After enough valid result data exists	Check calculation accuracy and avoid leaking student data.
P2	Shared Socket.IO adapter/background jobs/outbox	When load tests demonstrate need	Cross-instance correctness, retries, and observability.
P2	Camera/microphone proctoring	Separate approval only	Privacy impact, consent, minimization, retention, access controls, alternatives.
P2	Dedicated Android application	After web MVP is stable	Reuse APIs/domain rules; prove native capability is needed.

Feature prioritization rule

Every new feature should have a user problem, success criterion, dependency list, test plan, privacy/security review where applicable, and an explicit release target. If it does not improve the MVP's stated acceptance criteria, keep it in the backlog.

12. Keeping the handbook accurate over time

After a work session

Update the spreadsheet's Roadmap status, evidence/notes, and completion date. Record blockers instead of silently skipping them.

After a phase

Verify the phase exit gate, record the commit/test evidence, and only then begin the next phase.

When a design changes

Update the relevant requirements, architecture, database/API documentation, and Decision Log. Revisit affected task acceptance gates.

When adding a feature

Add it to Feature Backlog with priority and release target. Do not silently expand the MVP.

When preparing a new PDF

Use the latest tracker as the source of truth, preserve completed states, add the new work, and increment the handbook version/date.

When sharing evidence

Use redacted screenshots, test output, or commit references. Do not upload real student personal data or secrets into the project record.

Companion workbook tabs

Sheet	Purpose
Dashboard	Overall completion formulas and usage rules.
Roadmap	110 task-level milestones with folders, test instructions, acceptance gates, status, evidence, and dates.
Phase Gates	One explicit exit gate for every phase.
File Structure	Suggested repository layout and when each part is introduced.
Test Plan	24 baseline functional, security, presence, reliability, and privacy scenarios.
Feature Backlog	Prioritized MVP and later features.
Decision Log	Stack and product decisions, reasons, revisit triggers.

Final reminder

The project is not complete because the checklist is full. It is complete when the relevant acceptance gates pass, the behaviour is tested, the security limitations are documented, and the release can be reproduced.